Streaming Anomaly Detection and Drain Anticipation on Real HPC Telemetry

Tommaso Crippa

***Abstract*—I build an end-to-end streaming pipeline that replays real telemetry from the Marconi100 supercomputer through Kafka, aggregates it with Spark Structured Streaming into TimescaleDB, and scores nodes with pluggable online detectors. The pipeline sustains 74 k messages/s with a median end-to-end latency of 3.3 s and loses no data across a mid-run restart. Using 146 nodes over 2.5 years, I study two tasks against the Nagios label "node drained by the scheduler": detecting the current state and anticipating drains. Generic unsupervised detectors perform poorly (best per-node AUC 0.59). Drained nodes are mostly idle (73% of drained windows against 6% of normal ones), but idle nodes are rarely drained (21%). A single feature chosen on training data reaches 0.82 AUC and a supervised model 0.88, yet precision stays near 33%. For anticipation one hour ahead, a gradient-boosted model learned from delayed labels and retrained every four weeks matches the best single-feature rule in AUC but raises about four times fewer false alerts at equal recall, while drift-triggered retraining does not beat a fixed schedule.**

***Index Terms*—HPC monitoring, stream processing, anomaly detection, anomaly anticipation, concept drift, Kafka, Spark Structured Streaming**

I. Introduction

Large supercomputers are monitored continuously, and operators care most about whether each compute node is available. A node that the scheduler drains (for a fault, a configuration problem or maintenance) cannot run jobs until it is returned to service. Detecting such states, and ideally anticipating them, from the telemetry the machines already produce is therefore a recurring goal of operational data analytics [1], [2].

Most published detectors are evaluated offline on batches of data, with protocols that vary between papers, and rarely against trivial baselines. Streaming deployments are usually described as systems, without a statistical evaluation of what the models achieve. This report puts both in one place: a complete streaming pipeline built on Kafka and Spark [3], [4], run on real telemetry from CINECA's Marconi100 [5], and evaluated carefully on a long history of the same machine.

I ask three research questions:

- **RQ1** Can a streaming pipeline process real HPC telemetry reliably and with low latency?
- **RQ2** Can unsupervised detectors find drained nodes?
- **RQ3** Can drains be anticipated in a streaming setting?

The contributions are: (i) an open pipeline with idempotent aggregation, two independent consumers of one topic and a pluggable detector interface that runs unchanged in the stream and in batch mode; (ii) an evaluation on 146 nodes over 2.5 years with independent events, bootstrap intervals and paired tests, including trivial baselines that are chosen on training data only; (iii) the finding that the label is only weakly explained by telemetry beyond idleness, which explains why generic unsupervised detectors fail and limits what supervised models achieve.

Scope. The public data are 15-minute aggregates (four points per hour and node), so this is not a high-frequency sensor stream. I use the stream to exercise a realistic architecture, not to claim sub-minute detection.

II. Related Work

Dataset. M100 ExaData [5] is a public collection of Marconi100 monitoring data. I use its per-node, 15-minute aggregated release with a Nagios-derived label. The authors report an AUC of 0.57 for one node with a recurrent autoencoder.

Anomaly detection on HPC telemetry. Borghesi et al. [6] train autoencoders on normal behaviour of each node. RUAD [1] replaces the dense autoencoder with an LSTM encoder, trains without labels, and reports an AUC of 0.767 on all 980 nodes using 462 features that include CPU, memory and network load and labels cleaned with operator tickets. Neither the cleaned labels nor the system-level metrics are in the public release.

Anticipation. GRAAFE [2] predicts whether a node will be drained within a future window, with a rack-level graph neural network that reaches an AUC of 0.91 at one hour and 0.78 at six hours on the public data, against 0.61–0.64 for per-node baselines at one hour. I follow its label definition and evaluation protocol (section IV-A).

Streaming analytics. Rettig et al. [7] compute relative entropy and Pearson correlation over Kafka and Spark to detect anomalies in telecom streams. I adapt both to numeric telemetry. Half-space trees [8] are the standard streaming anomaly detector and are available in CapyMOA [9] and River [10]. For concept drift I use ADWIN [11], DDM [12] and OPTWIN [13].

This work differs in evaluating a deployed streaming path end to end, in reporting trivial baselines selected without test data, and in using paired comparisons that separate real gains from node-to-node noise.

III. Method

A. Streaming pipeline

Figure 1 shows the architecture. An ETL step converts each node's wide Parquet rows (one per 15 minutes, with avg/std/min/max of every metric and the Nagios label) into long events $(\text{node}, t, \text{metric}, \text{stat}, \text{value}, \text{state}, \text{anomaly flag})$. The metric list is configuration only; I use 12 metrics $\times$ 4 statistics = 48 values per node and window. A producer replays all nodes merged in event-time order at a configurable rate, serialises with Avro and a schema registry, and keys every message by node so that one partition preserves each node's order. The Kafka record timestamp is the publish time, which makes end-to-end latency measurable.

Aggregation. A Spark Structured Streaming job decodes the Confluent wire format (it strips the 5-byte header and fails if the schema id is not the expected one) and computes one-hour tumbling windows per (node, metric, stat) in event time, with a 30-minute watermark, i.e. two source samples. Results are written in update mode: every micro-batch is bulk-loaded into a staging table and merged with `INSERT ...ON CONFLICT DO UPDATE`, so a restart that replays a batch produces the same rows.

Detection. The detector service is a second consumer group on the same topic. It rebuilds the 48-value vector of each node and timestamp from the individual events, releasing a timestamp when all nodes are complete, when a node has moved three windows past it, or when a node has been silent for 30 s, and forward-fills values missing from late events. Detectors score each vector and then learn from it (prequential evaluation); scores go to TimescaleDB and flags to a second topic.

B. Detectors

All detectors implement one interface, `step(vectors, states)` $\rightarrow$ scores, and run unchanged in the Kafka service and in a batch mode that reads Parquet (used for the 2.5-year evaluation). Each node has its own model and its own online threshold.

- **Half-space trees** [8] from CapyMOA and from River (10 trees, depth 8, window 96), on min-max scaled features, with a variant that feeds each node's deviation from the cluster median.
- **Relative entropy and Pearson correlation** after Rettig et al. [7]: the Kullback–Leibler divergence between value histograms of adjacent 16-window blocks, and the correlation of a node with the cluster median. I adopt the paper's thresholds (a multiple of the baseline mean, and mean plus $k\sigma$).
- **RUAD replication**: a per-node LSTM autoencoder (window of 10) on all 352 aggregates, trained without labels.

Thresholds of every detector are also calibrated on a validation period for the F1 comparison. I add three references: *always OK*, the best *single-feature rule*, and a supervised *gradient-boosted trees* (GBT) model [14], [15] trained on the current label as a ceiling.

TABLE I
Nagios state of the scheduler-availability check.

State	Meaning
0	OK: the node can accept jobs
2	Critical: drained or unavailable to the scheduler (the anomaly label)
3	Unknown: the check could not determine the state; 39 timestamps, only during cluster-wide events, counted as anomalous
missing	Unmonitored: no label, excluded

C. Drain anticipation

Following GRAAFE [2], a node at time t is positive when it is drained in $(t, t{+}h]$ with $h \in \{1\,\text{h}, 6\,\text{h}\}$; rows whose previous row is already positive are removed, so only drains *entering* the horizon count (fig. 2). Features are the current 48 values, their changes over 1 h and 6 h, and deviations from the cluster and rack medians. The model is a histogram-based GBT trained on all positives and 5% of negatives.

In the stream, features seen at t are kept until $t{+}h$, when the live Nagios states reveal the label, and the example is then learned. I compare four policies for refitting the GBT on all resolved examples so far: never, every four weeks, and when a drift detector (ADWIN, DDM or OPTWIN) fires on the class-balanced prequential error, at most once a week. Incremental trees (River's adaptive random forest and Hoeffding adaptive tree, CapyMOA's adaptive random forest) serve as pure online alternatives.

IV. Results

A. Experimental setup

Data and labels. I use the per-node release of M100 ExaData [5]: one Parquet file per node with 15-minute windows, the average, standard deviation, minimum and maximum of each IPMI metric, and a Nagios label from the scheduler-availability check, which reports whether the scheduler can place jobs on the node (table I). The check runs every 15 minutes and always records a state: 87.2% of the 12.35 M node-windows in the evaluation set carry an explicit OK, 2.0% critical and 0.04% unknown. The remaining 10.7% have no label. They occur in whole blocks (March–April 2020 entirely, 91% of March 2021) and are excluded from every metric.

Slices. Table II lists the data used. The streaming replay covers four weeks of the three downloaded racks. For evaluation over a long history I read only the label columns of all 49 racks (857 nodes) with HTTP range requests and kept the 8 racks with the most independent events, 146 nodes in total. These racks were selected on label counts over the whole history, so event rates are higher than for a random rack.

Events and metrics. An *event* is a run of at least four consecutive drained windows on a node preceded by a normal window; onsets at the same timestamp within one rack count once. An onset is *cluster-wide* when at least half of the eligible nodes have an onset at the same window, and *node-specific* otherwise; 70% of the 1 h test onsets are cluster-wide. I report

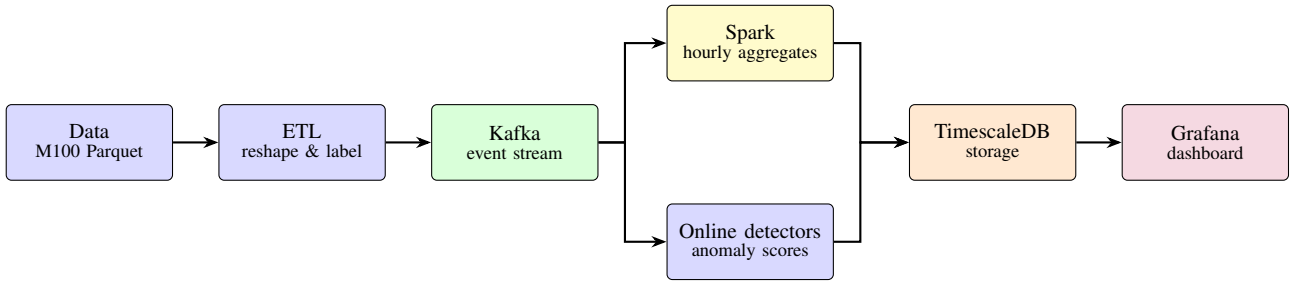


Fig. 1. Architecture. Two independent consumer groups read the same topic. Spark computes event-time aggregates and the detector service scores nodes; both write to TimescaleDB, which Grafana reads.

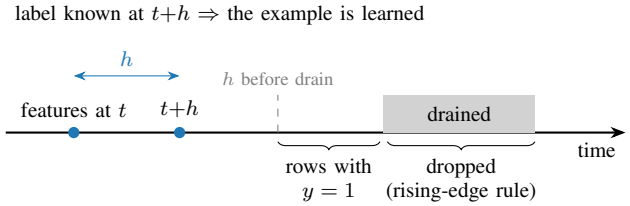


Fig. 2. Anticipation task and delayed labels. The label of the row at t is only known h windows later; the streaming model learns each example at that point.

TABLE II
DATA SLICES. WINDOWS ARE MONITORED NODE-WINDOWS.

Use	Content
Stream replay	49 nodes (racks 10–12), 2020-06-14 to 2020-07-12, 6.32 M events (48 values per node and window)
Detection	146 nodes. Train 2020-03 to 08, validation (thresholds) 2020-09 to 12, test 2021-01 to 2022-09: 8.01 M windows, 2.16% drained, 517 independent events
Anticipation	146 nodes, chronological 80/20 split, test 2022-03-26 to 2022-09-28: 1,313 onsets at 1 h among 2.54 M rows (base rate 0.05%), 1,242 at 6 h

ROC-AUC pooled over nodes and as the per-node median or mean, PR-AUC, precision and F1 at the threshold with the best validation F1, and for anticipation the number of false alerts per node-day at a fixed recall. Intervals are bootstrap 95% intervals over nodes. Models are compared with *paired* per-node AUC differences, which cancel node-to-node variation.

Single-feature rule. As a training-free baseline I score each window by one raw feature. The feature and its sign are chosen *on training data only*, among all 48 features, by mean per-node AUC; this picks `−p0_mem_power_std` for detection, `−total_power_max` for anticipation at 1 h and `−gpu0_core_temp_avg` at 6 h. A rule that I first hand-picked (−total power) after inspecting all data scores 0.728 and 0.654 at 1 h and 6 h, slightly better than the selected one; it is shown separately as an optimistic reference.

B. Pipeline (RQ1)

Table III reports three replays of the event stream on one machine (single broker, Spark in local mode, 5-second micro-batches), each on a fresh stack. The producer sustains 74 k messages/s. Spark keeps up at every rate: a micro-batch of ∼380 k rows takes 1.9 s, results land in TimescaleDB 3.3 s (median) and 5.9 s (p95) after publish, and the job finishes within 3–5 s of the producer. Most of that latency is the 5-second trigger interval.

The Python detector service is the bottleneck: its lag is 0.8 s at 12 k msg/s but grows to 14 s at 47 k and 76 s at 74 k msg/s, identically for a half-space-tree and a label-based detector, so the cost is decoding and vector reassembly, not scoring. The topic has eight partitions, so several detector instances could share the nodes.

Restart. I killed Spark about 75 s into a full replay and restarted it 15 s later. It resumed from its checkpoint, worked off the backlog in 500 k-row batches and, compared with an independent recomputation, produced exactly the expected 1,580,544 hourly windows with no missing, extra or duplicated key and no differing count, average, minimum or maximum.

C. Detecting drained nodes (RQ2)

Table IV compares generic unsupervised detectors with the references on the test period. All six unsupervised detectors are poor: per-node median AUC at most 0.59, pooled AUC between 0.34 and 0.51, F1 at most 0.055. Relative entropy and Pearson, with Rettig et al.'s thresholds, flag 90–98% of all windows. Half-space trees from CapyMOA are the best, and the peer-relative variants do not help.

Every unsupervised detector is clearly worse than the single-feature rule in the paired test, by 0.25 to 0.39 per-node AUC (winning on 0–3% of nodes). The supervised GBT is better than the rule by +0.047 [+0.039, +0.055], winning on 90% of nodes. This holds against the stronger hand-picked power rule as well (+0.04): the GBT wins on all 8 racks and in 15–16 of 20 test months, and three training seeds differ by 0.002.

Why. Figure 3 shows that 73% of drained windows sit at idle power, against 6% of normal windows. The converse is weak: of all idle windows only 21% are drained, about ten times the base rate but far from decisive, and 79% are nodes that are available and simply have no jobs. Power ranks windows well but cannot classify them: even the supervised GBT reaches

TABLE III
THROUGHPUT AND LATENCY AT THREE REPLAY RATES (FRESH STACK PER RUN). DETECTOR LAG IS SCORING TIME MINUS PUBLISH TIME.

Replay	msg/s	Batch (s)	End-to-end p50/p95 (s)	Detector lag p50 (s)
Max speed, 6.3M events	74k	1.9	3.3 / 5.9	75.6
20 windows/s, 1.6M	47k	1.7	3.7 / 6.0	14.2
5 windows/s, 1.6M	12k	0.9	3.2 / 5.4	0.8

TABLE IV
DETECTION OF THE CURRENT STATE, TEST 2021-01 TO 2022-09. AUC IS POOLED, PER-NODE AUC IS THE MEDIAN OVER 146 NODES; F1 USES THE BEST VALIDATION THRESHOLD.

Method	AUC	Per-node AUC	PR-AUC	F1
Always OK (flag nothing)	0.500	0.500	0.022	0.000
Best single feature, chosen on training data (-p0_mem_power_std)	0.818	0.819	0.089	0.175
Supervised GBT (ceiling)	0.881	0.859	0.372	0.362
HST (CapyMOA)	0.447	0.592	0.024	0.055
HST (River)	0.338	0.454	0.017	0.041
HST, peer-relative (CapyMOA)	0.393	0.457	0.018	0.040
HST, peer-relative (River)	0.363	0.426	0.019	0.038
Relative entropy	0.448	0.503	0.019	0.042
Pearson vs peers	0.514	0.455	0.026	0.041

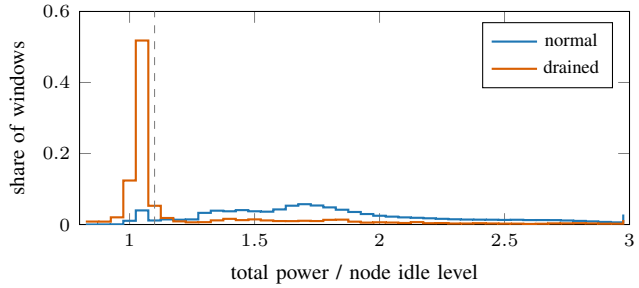


Fig. 3. Power relative to each node's idle level (10th percentile of training windows) in normal and drained test windows; the last bin collects values above 3. The dashed line marks the idle band (110%).

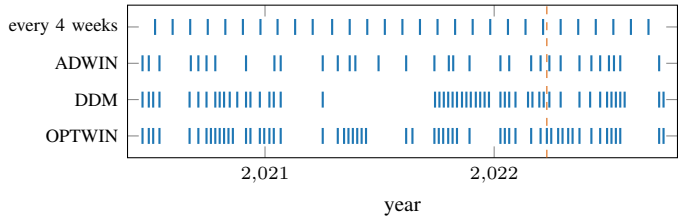


Fig. 4. Retraining times of the four policies. Drift detectors fire in bursts, mostly around maintenance events, and more often than the schedule without a gain.

only 33% precision at 39% recall. Density-based detectors rank *busy* windows, which are heterogeneous, as more unusual than the dense idle mode where drained nodes sit.

RUAD replication. A per-node LSTM autoencoder on all 352 aggregates of 32 random nodes scores a pooled AUC of 0.569 (per-node mean 0.606), which reproduces the 0.57 that the dataset authors report for this public release and is far below the 0.767 of the original paper, which uses non-public features and cleaned labels. On the same nodes and windows, CapyMOA half-space trees score 0.726, but across all 146 nodes they score 0.45 pooled (0.59 per-node median), so a sample of 32 nodes is misleading.

D. Anticipating drains (RQ3)

Table V gives the main results at 1 h and 6 h with the paired difference to the training-selected rule.

Baselines. The rule reaches 0.721 at 1 h and 0.637 at 6 h. A GBT given only total-power statistics does worse (0.695), a sign of the shift between the training and test years.

Offline model. GBT on telemetry and peer deviations reaches 0.739 [0.726, 0.753] at 1 h, better than the rule by +0.018 [+0.004, +0.032], and 0.665 at 6 h (+0.028 [+0.007, +0.049]). Its clearest advantage is at the top of the ranking: to catch 25% of onsets one hour ahead it raises 1.06 false alerts per node-day against 4.90 for the rule (paired difference +3.84 [+2.17, +4.22]), i.e. a precision of 1.2%, 23 times the base rate.

Streaming. Never retrained, the model is worse than the rule (0.604; 13 alerts). Refit every four weeks it reaches 0.717, statistically tied with the rule in AUC (−0.005 [−0.017, +0.007]) but with 1.15 alerts per node-day (+3.75 [+2.05, +4.12]). Triggering the refit with ADWIN, DDM or OPTWIN (0.729, 0.724, 0.723) gives the same AUC and 1.3–1.4 alerts, with 33, 52 and 58 retrains against 29 for the schedule (fig. 4): drift detection does not beat a fixed schedule here. Incremental trees fail (AUC 0.33–0.53).

Node-specific onsets. Removing cluster-wide onsets leaves 392 (1 h) and 321 (6 h) onsets. Here no learned model is better than the rule: the offline model scores 0.710 against 0.731 at 1 h (−0.020 [−0.054, +0.016]) and 0.562 against 0.667

TABLE V

ANTICIPATION OF DRAINS WITHIN 1 H AND 6 H. AUC IS THE MEAN PER-NODE AUC WITH BOOTSTRAP 95% INTERVAL; Δ IS THE PAIRED DIFFERENCE TO THE BEST SINGLE-FEATURE RULE[a] (ALL ONSETS, 1 H); ALERTS ARE FALSE ALERTS PER NODE-DAY AT 25% RECALL. [a] CHOSEN ON TRAINING DATA (−TOTAL_POWER_MAX AT 1 H, −GPU0_CORE_TEMP_AVG AT 6 H). [b] CHOSEN AFTER INSPECTING ALL DATA, OPTIMISTIC.

Model	1 h AUC	1 h node-spec.	Δ vs rule	1 h alerts	6 h AUC	6 h alerts
Always OK	0.500 [.500,.500]	0.500 [.500,.500]	–	96.0	0.500 [.500,.500]	96.0
Best single feature (train)[a]	0.721 [.707,.735]	0.731 [.690,.769]	–	4.9	0.637 [.621,.652]	11.2
Low power, hand-picked[b]	0.728 [.713,.741]	0.736 [.697,.772]	–	3.6	0.654 [.638,.669]	6.5
GBT on power only	0.695 [.680,.708]	0.689 [.653,.726]	–	5.1	0.621 [.607,.635]	7.5
GBT, telemetry + peers (offline)	0.739 [.726,.753]	0.710 [.670,.745]	+0.018	1.1	0.665 [.651,.678]	5.3
GBT, all features (offline)	0.729 [.717,.740]	0.675 [.643,.707]	+0.007	3.9	0.674 [.661,.688]	12.2
Streaming GBT, never retrained	0.604 [.587,.620]	0.594 [.557,.632]	-0.117	13.0	–	–
Streaming GBT, every 4 weeks	0.717 [.702,.731]	0.714 [.675,.749]	-0.005	1.2	–	–
Streaming GBT, ADWIN	0.729 [.712,.744]	0.705 [.665,.744]	+0.008	1.3	–	–
Streaming GBT, DDM	0.724 [.708,.738]	0.694 [.656,.730]	+0.003	1.3	–	–
Streaming GBT, OPTWIN	0.723 [.707,.737]	0.688 [.648,.723]	+0.001	1.4	–	–
Incremental trees, River ARF	0.478 [.464,.492]	0.597 [.556,.641]	–	28.0	–	–
Incremental trees, River HAT	0.332 [.322,.343]	0.393 [.352,.435]	–	30.1	–	–
Incremental trees, CapyMOA ARF	0.527 [.518,.537]	0.610 [.579,.640]	–	96.0	–	–

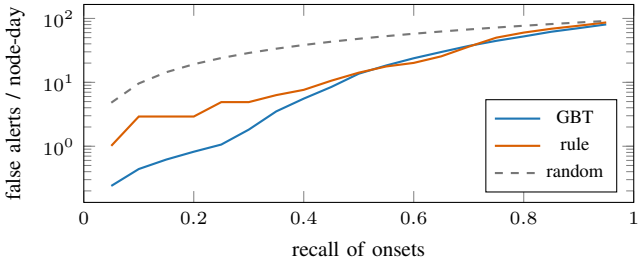


Fig. 5. False alerts per node-day needed to reach a given recall of 1 h onsets. The rule's curve is flat at low recall because many idle windows tie at the same value.

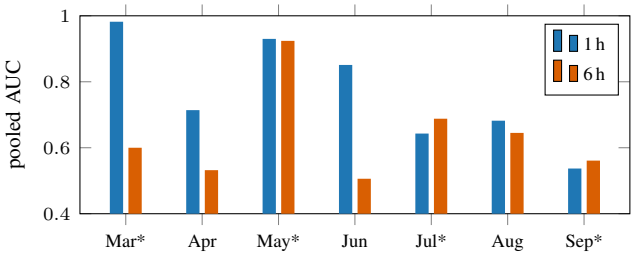


Fig. 6. Pooled AUC of the offline GBT per test month (2022). Asterisks mark months in which at least half of the onsets belong to cluster-wide events.

at 6 h (−0.106 [−0.153, −0.058]) (table VI lists the other feature sets). Calendar features alone fall below chance for node-specific onsets (0.373 at 6 h), consistent with learning the maintenance schedule.

Operating point and stability. Figure 5 shows the cost of recall: half of the onsets cost about 14 alerts per node-day (precision 0.18%), so the model is a risk ranking rather than an alarm. Results also depend strongly on which events fall in the test period (fig. 6): the pooled AUC of the offline model at 1 h ranges from 0.54 to 0.98 across months, and months dominated by cluster-wide drains are not uniformly easy (0.98 and 0.93 in March and May, 0.64 and 0.54 in July and September).

V. DISCUSSION

What the label measures. The Nagios state records a scheduler decision, not a telemetry anomaly. Drained nodes are almost always idle, but most idle nodes are available, so the information that separates the two lies outside the node's own metrics (reservations, maintenance windows, administrative action). That caps what telemetry can do: even a supervised model with all telemetry and peer deviations reaches 33% precision. Earlier work reports higher numbers when labels are cleaned with operator tickets and when system-level load metrics are available [1]; my replication on the public release shows that neither is in the public data.

Unsupervised detection (RQ2). Generic detectors score deviation from the dense regime, and the regime that contains drained nodes is the densest one, so tuning cannot fix them. The "low power" rule beats them by a wide margin without any training.

Anticipation (RQ3). Learned models do not improve overall AUC much over the best single feature, and are no better for node-specific drains, but they improve the top of the ranking: about four times fewer false alerts at 25% recall one hour ahead. Retraining is necessary (a model that is never refitted is worse than the rule), yet a fixed schedule is as good as drift detectors, which on rare, bursty positives mostly react to maintenance events.

Threats to validity. (i) One test period of six months: intervals are over nodes and do not cover other periods, and monthly AUC varies from 0.54 to 0.98. (ii) Racks were selected for many events, and the headline feature set was selected among feature sets compared on the test period, so the offline row is mildly optimistic; the streaming policies were

TABLE VI
OFFLINE GBT BY FEATURE SET: MEAN PER-NODE AUC FOR ALL AND NODE-SPECIFIC ONSETS.

Features	1 h all	1 h node-spec.	6 h all	6 h node-spec.
Telemetry + peers	0.739 [.726,.753]	0.710 [.670,.745]	0.665 [.651,.678]	0.562 [.524,.598]
All features	0.729 [.717,.740]	0.675 [.643,.707]	0.674 [.661,.688]	0.449 [.413,.486]
Telemetry only	0.717 [.703,.732]	0.711 [.669,.745]	0.607 [.590,.623]	0.585 [.549,.620]
Calendar only	0.537 [.529,.544]	0.502 [.472,.533]	0.565 [.552,.580]	0.373 [.331,.414]
Rack context only	0.508 [.498,.518]	0.489 [.467,.511]	0.497 [.491,.503]	0.511 [.492,.527]

not tuned. (iii) The alert thresholds are read off test scores. (iv) The data are 15-minute aggregates from one machine, with one broker and Spark in local mode in the pipeline measurements.

Future work. Heavy-tailed, per-node lateness with outages and a watermark sweep would show the completeness/state trade-off, which the replay only exercises with small fixed delays. Synthetic anomalies with known onsets would complement the scheduler label, and graph models over the rack layout [2] are the obvious next detector.

VI. CONCLUSION

The Kafka–Spark pipeline replays and aggregates real HPC telemetry at 74 k messages/s with seconds of latency and restarts without losing or duplicating data (RQ1). Generic unsupervised detectors miss drained nodes because they sit in the dense idle mode; a single feature chosen on training data and a supervised model both do far better, but precision is bounded at about one third because most idle nodes are not drained (RQ2). Drains can be anticipated one hour ahead with a streaming model learned from delayed labels and retrained on a schedule; it matches the best single-feature rule in AUC and needs about four times fewer false alerts at equal recall, but is no better for node-specific drains (RQ3).

REFERENCES

[1] M. Molan, A. Borghesi, D. Cesarini, L. Benini, and A. Bartolini, "RUAD: Unsupervised anomaly detection in HPC systems," *Future Generation Computer Systems*, vol. 141, 2023.

[2] M. Molan, M. Seyedkazemi Ardebili, J. A. Khan, F. Beneventi, D. Cesarini, A. Borghesi, and A. Bartolini, "GRAAFE: GRaph anomaly anticipation framework for exascale HPC systems," *Future Generation Computer Systems*, vol. 160, pp. 644–653, 2024.

[3] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," in *NetDB Workshop*, 2011.

[4] M. Armbrust *et al.*, "Structured streaming: A declarative API for real-time applications in Apache Spark," in *ACM SIGMOD International Conference on Management of Data*, 2018.

[5] A. Borghesi *et al.*, "M100 ExaData: a data collection campaign on the CINECA's Marconi100 Tier-0 supercomputer," *Scientific Data*, vol. 10, 2023.

[6] A. Borghesi, A. Bartolini, M. Lombardi, M. Milano, and L. Benini, "A semisupervised autoencoder-based approach for anomaly detection in high performance computing systems," *Engineering Applications of Artificial Intelligence*, 2019.

[7] L. Rettig, M. Khayati, P. Cudré-Mauroux, and M. Piórkowski, "Online anomaly detection over big data streams," in *IEEE International Conference on Big Data*, 2015.

[8] S. C. Tan, K. M. Ting, and T. F. Liu, "Fast anomaly detection for streaming data," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2011.

[9] H. M. Gomes, A. Lee, N. Gunasekara, Y. Sun, G. W. Cassales, J. J. Liu, M. Heyden, V. Cerqueira, M. Bahri, Y. S. Koh, B. Pfahringer, and A. Bifet, "CapyMOA: Efficient machine learning for data streams in Python," *arXiv preprint arXiv:2502.07432*, 2025.

[10] J. Montiel *et al.*, "River: machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[11] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *SIAM International Conference on Data Mining*, 2007.

[12] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence (SBIA)*, 2004.

[13] M. D. L. Tosi and M. Theobald, "OPTWIN: Drift identification with optimal sub-windows," in *IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2024, pp. 331–337.

[14] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[15] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.